Django + DRF Day 1 Study Notes

Chapter 1 - Introduction

Q1. What is Django?

Django is a high-level Python web framework used to build websites and web applications quickly by providing an ORM, authentication, URL routing, templates, and an admin panel.

Q2. What is DRF?

Django REST Framework extends Django to build REST APIs by handling JSON serialization, validation, authentication, permissions and CRUD.

Q3. What is a REST API?

A REST API allows applications to communicate over HTTP using JSON and methods like GET, POST, PUT, PATCH and DELETE.

Chapter 2 - Project Structure

What is a Project?

The entire Django application.

What is an App?

A self-contained feature/module inside the project.

Why separate them?

To keep code modular and reusable.

startproject

Creates the Django project.

startapp

Creates a Django app.

Chapter 3 - Important Files

manage.py

Runs Django commands.

settings.py

Project configuration.

project urls.py

Main URL router.

models.py

Defines database tables.

serializers.py

Converts Models ↔ JSON.

views.py

Handles requests.

admin.py

Registers models in Django Admin.

apps.py

App configuration.

tests.py

Stores tests.

migrations/

Database schema history.

wsgi.py

Production entry point.

asgi.py

Async entry point.

Chapter 4 - Models & ORM

Model

Blueprint of a database table.

Field

Represents one database column.

CharField

Stores short text.

BooleanField

Stores True/False.

DateTimeField

Stores date and time.

auto_now_add=True

Automatically stores creation timestamp.

__str__()

Human-readable representation.

ORM

Lets Python communicate with SQL.

Manager (objects)

Database manager.

QuerySet

Collection of queried objects.

Task.objects.all()

Returns every Task.

Chapter 5 - Serializers

Serializer

Converts Python objects ↔ JSON.

ModelSerializer

Automatically creates serializer fields.

Meta

Configuration class.

model = Task

Selects the model.

fields='__all__'

Includes all fields.

Chapter 6 - Views

View

Handles HTTP requests.

ViewSet

Groups related API operations.

ModelViewSet

Automatic CRUD operations.

queryset

Records used by the ViewSet.

serializer_class

Serializer blueprint.

Chapter 7 - URLs

path()

Maps URLs.

include()

Imports another urls.py.

DefaultRouter

Generates REST endpoints.

router.register()

Registers a ViewSet.

Chapter 8 - Database Commands

makemigrations

Creates migration files.

migrate

Applies migrations.

runserver

Starts the development server.

Chapter 9 - HTTP Methods

GET

Read data.

POST

Create data.

PUT

Replace existing resource.

PATCH

Update selected fields.

DELETE

Delete data.

Chapter 10 - Request Flow

Flow

Browser → Project URLs → App URLs → ViewSet → QuerySet(Model) → Database → Serializer → JSON → Browser

Chapter 11 - Complete Workflow

Steps

1. Create Project
2. Create App
3. Add app to INSTALLED_APPS
4. Create Model
5. makemigrations
6. migrate
7. Create Serializer
8. Create ModelViewSet
9. Create Router
10. Include app URLs
11. runserver
12. Test API

Code Written

Model

class Task(models.Model):

title=CharField(...)

completed=BooleanField(...)

created_at=DateTimeField(...)

__str__()

Serializer

class TaskSerializer(ModelSerializer):

class Meta:

model=Task

fields='__all__'

ViewSet

class TaskViewSet(ModelViewSet):

queryset=Task.objects.all()

serializer_class=TaskSerializer

Router

router=DefaultRouter()

router.register('tasks',TaskViewSet)